Operating Systems Assignment 01

Custom Kernel Compilation and System Call Implementation

Roll Number: K240707

March 20, 2026

Contents

1 Prerequisites and Dependency Installation

Before beginning the kernel compilation process, it is essential to install the necessary dependencies. This ensures that the system has all the required compilers, libraries, and tools to build the kernel from source without failing midway.

```
sudo apt update
sudo apt install build-essential libncurses-dev bison flex libssl
   -dev libelf-dev -y
```

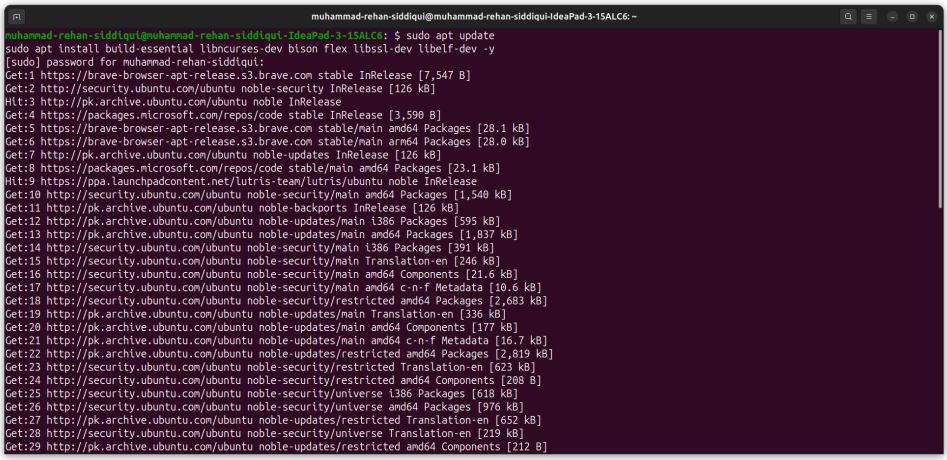

Figure 1: Installing system dependencies

2 Downloading and Extracting the Kernel Source

The next step is to acquire the raw Linux kernel source code. I downloaded kernel version 6.1.80 directly from the official Linux kernel archive and extracted the tarball into my local directory.

```
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.1.80.
   tar.xz
tar -xvf linux-6.1.80.tar.xz
cd linux-6.1.80
```

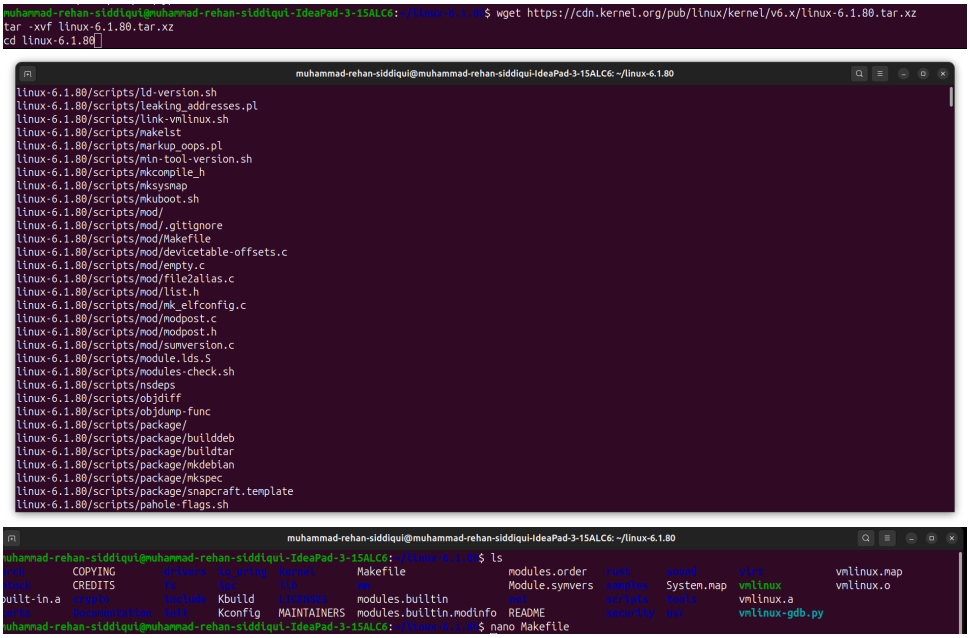

Figure 2: Extracting the downloaded kernel

3 Makefile Modification (Kernel Branding)

To prove that this kernel was custom-built for this assignment, I modified the main `Makefile` of the kernel source. I appended my roll number to the `EXTRAVERSION` variable so that it reflects in the system's kernel release name.

- **Modification:** Changed the line to `EXTRAVERSION = -K240707`

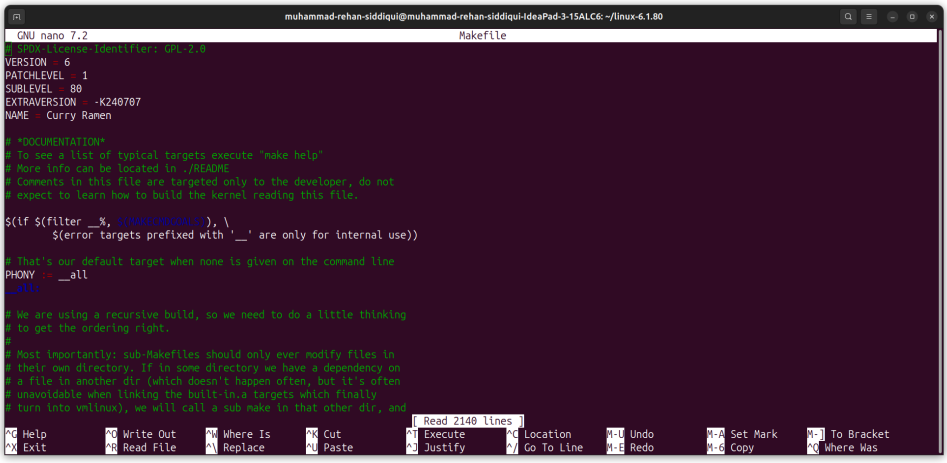

Figure 3: Modifying the EXTRAVERSION variable in the main Makefile

4 System Call Implementation

Implementing the custom system call involved modifying the kernel source code in four distinct areas:

4.1 Creating the System Call Logic

I created a new C file named `hello_syscall.c` inside the `kernel/` directory. This file contains the actual logic to print my custom message to the kernel log.

```
#include <linux/kernel.h>
#include <linux/syscalls.h>

SYSCALL_DEFINE0(hello) {
    printk("Hello World! system call for roll number K240707\n");
    return 0;
}
```

4.2 Linking the System Call

To ensure the compiler includes my new C file during the build process, I appended the object file to the `kernel/Makefile`.

```
obj-y += hello_syscall.o
```

4.3 Updating the System Call Table

I registered my system call in the 64-bit system call table (`arch/x86/entry/syscalls/syscall_64.tbl`) by assigning it the specific system call number 451.

```
451     common     hello     sys_hello
```

4.4 Updating the Header File

Finally, I declared the function prototype in the main system calls header file (`include/linux/syscalls`) just before the `#endif` directive.

```
asmlinkage long sys_hello(void);
```

Figure 4: Modifying the system call table and headers (Part 1)

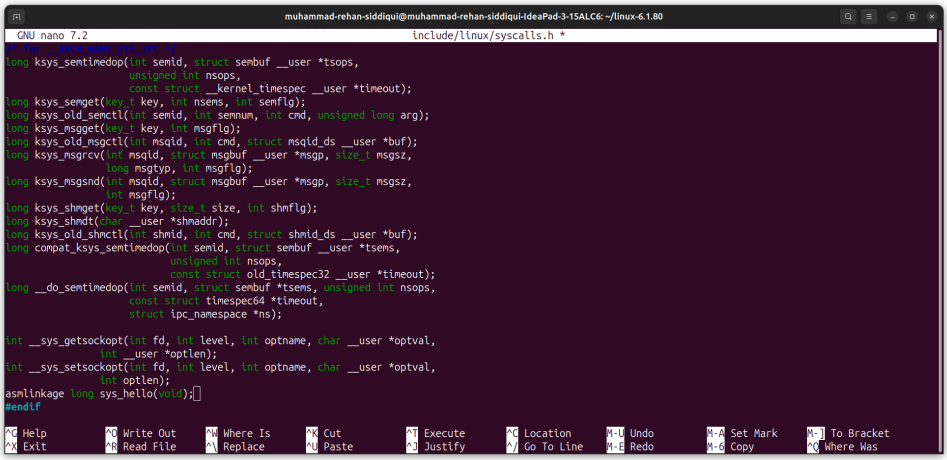

Figure 5: Modifying the system call table and headers (Part 2)

5 Kernel Configuration

Instead of writing a configuration from scratch, I copied the configuration file of my currently running system. This guarantees that the new kernel will have all the necessary drivers for my specific hardware. I then updated the configuration to the default settings for the new kernel version.

```
cp -v /boot/config-$(uname -r) .config
make olddefconfig
```

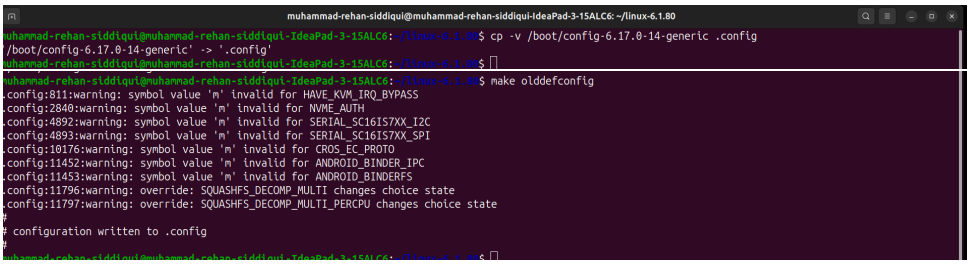

Figure 6: Copying and updating the kernel configuration

6 Bypassing Security Certificates

A common issue when compiling on Ubuntu is the build failing due to missing Canonical security keys. To bypass this, I utilized the kernel's configuration script to disable the requirement for trusted and revocation keys.

```
scripts/config --disable SYSTEM_TRUSTED_KEYS
scripts/config --disable SYSTEM_REVOCATION_KEYS
```

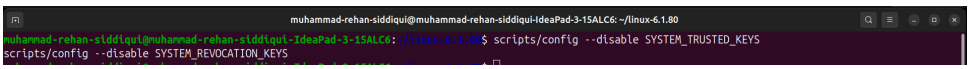

Figure 7: Disabling system trusted and revocation keys

7 Final Compilation

With the environment fully configured, the system call implemented, and the keys bypassed, I initiated the final compilation. I used the `-j$(nproc)` flag to utilize all available CPU cores for a faster build.

```
make -j$(nproc)
```

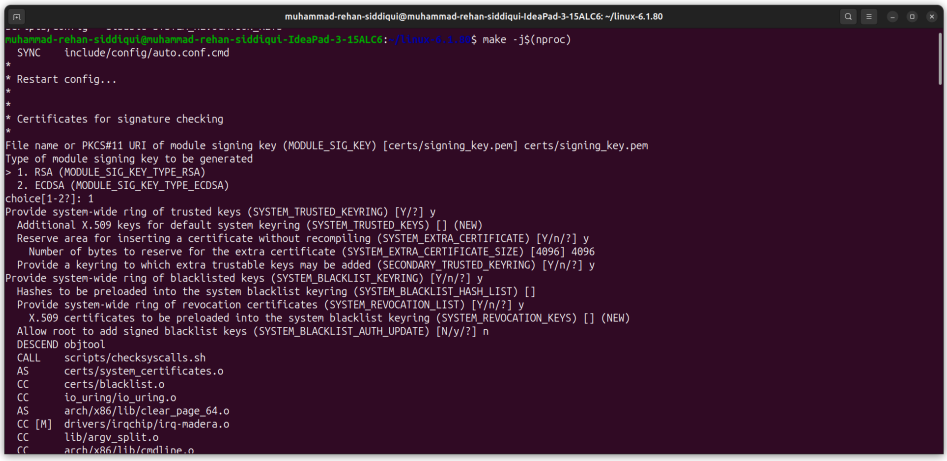

Figure 8: Executing the final compilation utilizing all CPU cores

8 Kernel Installation and System Reboot

After a successful compilation, the final phase is to install the compiled modules and the kernel image into the system's `/boot` directory. Finally, the GRUB bootloader must be updated to recognize the newly installed custom kernel before rebooting the system.

```
sudo make modules_install
sudo make install
sudo update-grub
sudo reboot
```

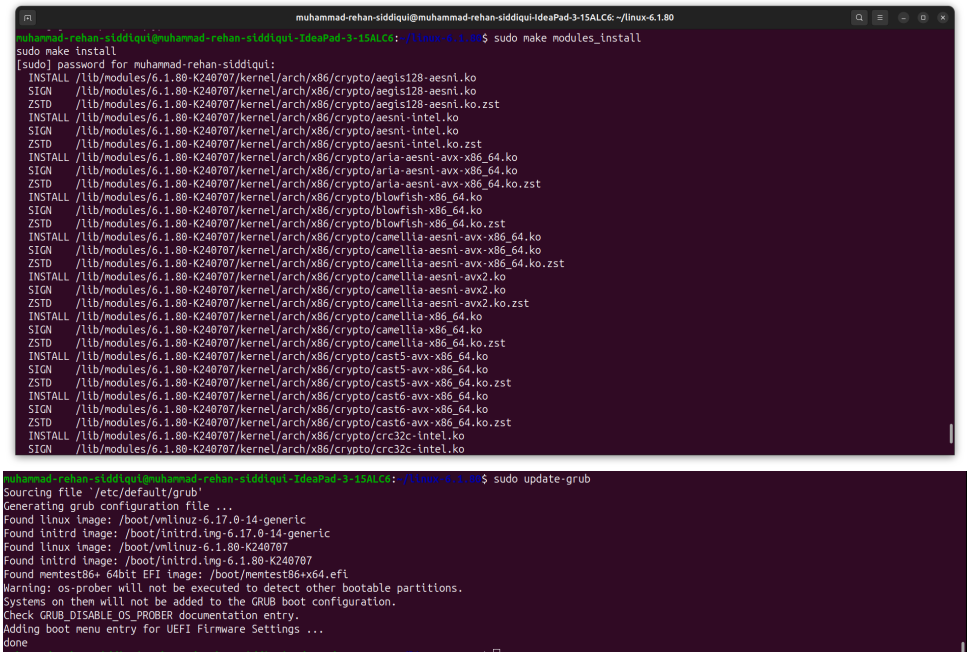

Figure 9: Installing modules, updating GRUB, and rebooting into the new kernel

9 Verification

After rebooting, the first step is to verify that the system is successfully running the newly compiled custom kernel. Then, a user-space C program is created to invoke the new system call. Finally, the kernel logs are checked to confirm the system call executed successfully and printed the correct message.

9.1 Verifying the Kernel Version

```
uname -r
# Expected Output: 6.1.80-K240707
```

9.2 Writing the Test Program (`task.c`)

I created a simple C program to test the newly added system call (number 451).

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>
int main() {
    long int check = syscall(451);
    if(check == 0) {
        printf("System call executed successfully!\n");
    } else {
        perror("Error");
    }
    return 0;
}
```

9.3 Executing and Checking Logs

I compiled and ran the test program, then checked the `dmesg` logs to verify that the kernel printed my roll number.

```
gcc task.c -o task
./task
sudo dmesg | grep "K240707"
```

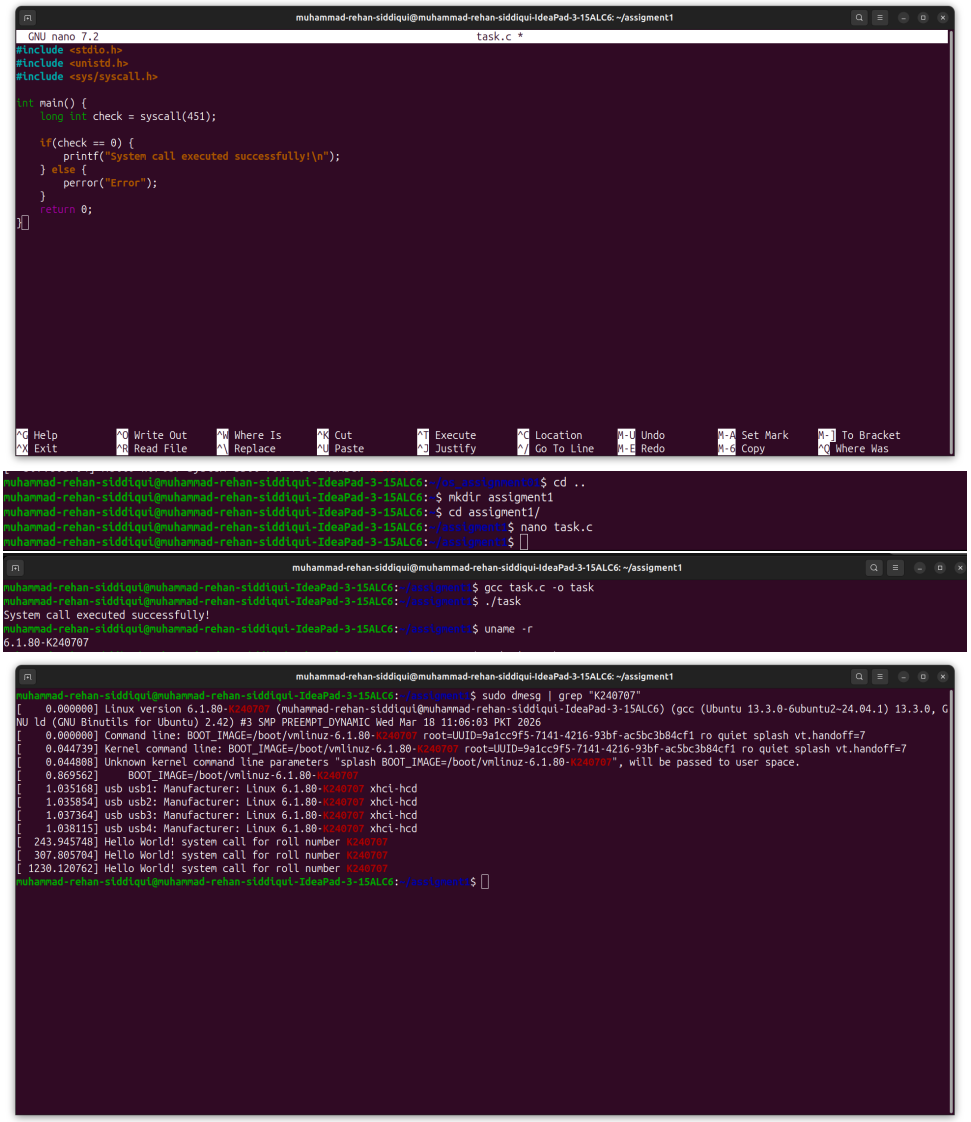

Figure 10: Verifying the custom kernel version and system call execution in dmesg